

Layout-Conditioned Flow Matching with Mask-Guided Region Pooling for Anime Face Generation: A Complete Record of Methods and Decisions

tinyflow project
feat/pixel-unet, September 2026

Abstract

We document, end to end, the construction of a 64×64 anime-face generator based on flow matching with x -prediction in pixel space, trained from scratch on 21,551 images with one RTX 4090. The record covers the data pipeline, the network to the level of every layer’s normalisation and initialisation, the objective, optimiser, sampler and evaluation protocol, and—at equal weight—every design decision, including the ones that failed. The central negative result is a derivation, confirmed by ablation, that no auxiliary loss on the predicted clean image against the target (edges, lines, palettes, mask-weighted variants, per-pixel re-weightings) can improve a flow-matching model: at the objective’s optimum such penalties are almost entirely irreducible and only bias the network toward hedged predictions. The central positive result is a structural change: conditioning on a three-channel semantic layout (face, eyes, mouth hulls from a landmark detector), sampled at test time from a Gaussian-mixture prior over a point-distribution model of the landmarks, together with *mask-guided region pooling*—a zero-initialised layer that pools decoder features inside each semantic region and broadcasts a projection back into that region only. It reduces the rate of mismatched iris colours from 34% to the data’s own 6% at slightly better FID. We also report where the model fails (coverage: recall 0.15 against a real-vs-real ceiling of 0.77), a sampling-time edge-guidance that recovers 3–8 FID without retraining, and the learning-rate instability that shaped the final training schedule.

1 Introduction

This paper is a methods record rather than a claim of state of the art. Its purpose is to let a reader reproduce every number, understand why each component is what it is, and avoid repeating experiments that were tried and closed. It accompanies the `tinyflow` repository; where a symbol is defined in code, the file and function are named.

The starting point was a working pixel-space flow-matching model that shipped with two domain-motivated auxiliary losses (a Sobel gradient-difference loss and a dark-line “ink” loss), the intuition being that anime is line art over flat fills. The campaign asked whether such dataset-specific inductive biases help. Answering that required (i) a theory of what an auxiliary loss can do at the flow-matching optimum (§5), (ii) an offline test-bed that does not need training (§6), (iii) paired ablations with a measured noise floor (§7), and (iv) a diagnosis of where the best model actually fails (§10). The diagnosis pointed at

two distinct problems—a long-range consistency failure (mismatched iris colours) and a coverage failure—and the first of these is solved by the architecture of §12.

Contributions, in the order they were established:

1. A derivation (§5) that training-time losses on $\hat{x}$ against x_1 are irreducible at the conditional-mean optimum, and that x_1 -dependent per-pixel weights tilt the learned field; both confirmed by ablation (§7).
2. A measurement protocol (§4): FID noise, loss as the sensitive statistic for architecture changes, a validated iris-mismatch metric, and a 16-step sampler for evaluation.
3. A layout prior over detector landmarks (§3.6) and a conditioned model with mask-guided region pooling (§3.5), the first structural change with an unambiguous, reproduced win.
4. A bottleneck analysis (§10) locating the remaining gap in coverage and in small low-variance features (mouth, nose).

2 Data

Images. 21,551 anime face crops at $64 \times 64 \times 3$ (`data/animefaces.py`). PNGs are loaded once, converted to `float32`, mapped to $[-1, 1]$ by $x/127.5 - 1$ and cached (`.preprocessed/anime_faces.npy`). All public boundaries use (H, W, C) layout; only the U-Net’s interior is channels-first. Images are denormalised for display with `clip((x + 1) · 127.5, 0, 255)`; clipping is mandatory because the network has no bounded output activation (§3.4).

Augmentation. A Grain pipeline applies, per image, on worker threads: a horizontal flip with probability 0.5 (applied jointly to the image and its layout mask), then colour jitter with brightness shift $b \sim U(-0.2, 0.2)$, contrast $c \sim U(0.8, 1.2)$, saturation $s \sim U(0.8, 1.2)$, as $x \leftarrow x + b$; $x \leftarrow \bar{x} + c(x - \bar{x})$ with $\bar{x}$ the image mean; $x \leftarrow g + s(x - g)$ with g the per-pixel channel mean; then clipped to $[-1, 1]$. Masks pass through the jitter untouched. Batch size 128; batches straddle epoch boundaries (ceiling division), so an epoch is $\lceil N/128 \rceil$ batches.

Landmarks and masks. Each image is upsampled to 256×256 (bicubic) and passed to `hysts/anime-face-detector` (YOLOv3 face detector, HRNet 28-keypoint regressor) with the whole image as the face box. The 28 points are fixed in number and order: 0–4 chin contour left to right; 5–7 and 8–10 the two brows left to right; 11–16 and 17–22 the two eyes, each as the upper lid left to right (3 points) then the lower lid left to right (3 points); 23 nose; 24–27 mouth (left corner, top, right corner, bottom). Coordinates are stored in the image’s $[-1, 1]^2$ frame with the pixel-centre convention $((p + 0.5)/256) \cdot 2 - 1$; points the detector places outside the crop are kept (they occur for tight crops). Per-point confidences $\in [0, 1]$ are stored alongside (`experiments/extract_landmarks.py`).

The three-channel mask used for conditioning is rasterised from the landmarks: channel 0 the convex hull of points 0–10 (face), channel 1 the union of the hulls of 11–16 and 17–22 (eyes), channel 2 the hull of 24–27 (mouth). Hulls are filled at 256×256 and area-averaged to 64×64 , giving soft, anti-aliased edges; values are stored as `uint8` and used in $[0, 1]$. The eye channel covers $\approx 4.8\%$ of pixels. `data/layouts.py:rasterize`

reproduces the stored masks with IoU 0.99 (face), 0.95 (eyes), 0.64 (mouth; a few-pixel region, quantisation).

Curation. Sorting images by mean landmark confidence and inspecting bands (`runs/ablation/curation_*`) below 0.3 the content is dominated by non-faces (torsos, clothing, hands, duplicated crops); the 0.3–0.65 band is legitimate hard faces (closed eyes, glasses, masks, mascots) that a generator should keep. Training therefore drops images with mean confidence < 0.3 (317 images, 1.47%; 21,234 remain, 166 batches per epoch). **FID reference statistics always use the full set**, so scores stay comparable across runs. Measured effect (§7): neutral to slightly positive.

3 Model

3.1 Flow matching with x-prediction

With $x_1 \sim p_{\text{data}}$, $x_0 \sim \mathcal{N}(0, I)$ and a time $t \in [0, 1]$, the interpolant is $x_t = t x_1 + (1 - t) x_0$ and the conditional velocity is $x_1 - x_0$. The network predicts the *clean image* $\hat{x} = f_\theta(x_t, t)$ (x-prediction), and the velocity is derived wherever it is needed as

$$u_\theta(x_t, t) = \frac{\hat{x} - x_t}{\max(1 - t, \epsilon_d)}, \quad \epsilon_d = 0.05. \quad (1)$$

The training loss is the plain conditional flow-matching loss in velocity space,

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_0, x_1} \frac{1}{2} \|u_\theta(x_t, t) - (x_1 - x_0)\|^2 = \mathbb{E} \frac{1}{2} \frac{\|\hat{x} - x_1\|^2}{\max(1 - t, \epsilon_d)^2}, \quad (2)$$

i.e. a per-pixel squared error on $\hat{x}$ with weight $w(t) = 1 / \max(1 - t, \epsilon_d)^2$ (≈ 1.2 at $t = 0.1$, 4 at 0.5, 100 at 0.9, 400 at the floor). The floor bounds the weight so that a single sample near $t = 1$ cannot own a batch’s gradient; this is the JiT recipe (Li & He, 2026). t is drawn logit-normal, $t = \sigma(\mu + \sigma_t z)$, $z \sim \mathcal{N}(0, 1)$, $\mu = -0.8$, $\sigma_t = 1$, which puts more mass at low t (high noise) and thins the tail at $t \rightarrow 1$ where $w(t)$ is large. The population minimiser of (2) at every (x_t, t) is the conditional mean $x^*(t) = \mathbb{E}[x_1 | x_t]$ (a fact used throughout §5).

The loss has no auxiliary terms. This was not the starting point; it is the conclusion of §5–7.

3.2 Conditioning token

For a conditioned model with $K = 3$ mask channels, the network input is the concatenation along channels

$$x_{\text{in}} = [x_t; m \odot \kappa; \kappa] \in \mathbb{R}^{H \times W \times (3+K+1)}, \quad (3)$$

where $m \in [0, 1]^{H \times W \times K}$ is the layout mask and $\kappa \in \{0, 1\}$ a per-sample indicator broadcast to a channel (`imagefm.cond_token`). The *null token* is $\kappa = 0$ with zero mask channels; the indicator disambiguates “unconditioned” from “a face with no parts”. Classifier-free training (Ho & Salimans, 2022) replaces the mask by the null token with probability p_{drop} (`cond_dropout`); the final recipe uses $p_{\text{drop}} = 0$ (a pure conditional model) because dropout cost ≈ 7 FID at the screening horizon (§12.1). The target is then $\mathbb{E}[x_1 | x_t, m]$: the layout is given and the network renders. Sampling never uses a real image’s mask; layouts come from the prior of §3.6.

3.3 Sampler

The probability-flow ODE $\dot{x} = u_\theta(x, t)$ is integrated from $t = 0$ to $t_1 = 1 - 10^{-3}$ with `diffraX`'s `Dopri5` on a fixed grid of n steps ($\Delta t = t_1/n$, no adaptive control), $n = 64$ for reported samples and $n = 16$ for FID during training and screening (§4). One solver (`imagefm._solve`) serves training-time samples, FID and figures. For a conditioned model the conditioning token is held fixed along the trajectory.

3.4 Network

The velocity network is a four-level U-Net (`models/unet.py`), channels-first internally, 36.7 M parameters at `base_channels=64` (9.38 M at 32).

- **Input/output.** A 3×3 convolution (padding 1) maps the $3 + K + 1 = 7$ input channels to 64; a 1×1 convolution maps the final 64 channels to the 3 output channels. There is **no final activation**: the velocity derived from $\hat{x}$ is unbounded regardless, and a bounded output would only add vanishing gradients.
- **Time embedding.** t is scaled by 10^3 and embedded with 64 frequencies ω_k geometrically spaced from 1 to 10^{-4} , $e(t) = [\sin(10^3 t \omega_k); \cos(10^3 t \omega_k)] \in \mathbb{R}^{128}$, then a two-layer MLP $128 \rightarrow 128 \rightarrow 128$ with SiLU. Without the 10^3 scale no frequency completes a cycle on $[0, 1]$ and the embedding is nearly constant in t .
- **ResBlock** (in c , out c'): GroupNorm(8 groups, $\epsilon = 10^{-5}$, statistics over the group's channels *and* space, learned affine) $\rightarrow$ SiLU $\rightarrow 3 \times 3$ conv $\rightarrow$ GroupNorm (no affine) $\rightarrow$ adaptive modulation $h \odot (1 + \gamma) + \beta$ with $[\gamma; \beta] = W_a \text{SiLU}(e) + b_a$, $W_a \in \mathbb{R}^{2c' \times 128}$ **zero-initialised** $\rightarrow$ SiLU $\rightarrow 3 \times 3$ conv, plus a skip path (1×1 conv if $c \neq c'$, identity otherwise). The zero initialisation follows adaLN-Zero (Peebles & Xie, 2023), the FiLM analogue of the zero- γ trick of Wu & He (2018); the modulation has no output gate. GroupNorm (Wu & He, 2018) rather than InstanceNorm is deliberate: per-channel statistics would strip per-image brightness and colour cast, which a generator must keep.
- **UBlock** at level i with channels $c_i = 64 \cdot 2^i$: *down* = ResBlock($c_i \rightarrow 2c_i$) $\rightarrow$ SiLU $\rightarrow$ depthwise 2×2 stride-2 conv; *up* = bilinear $\times 2 \rightarrow 3 \times 3$ conv ($2c_i \rightarrow c_i$) $\rightarrow$ SiLU $\rightarrow$ concatenate the skip (c_i channels) $\rightarrow$ ResBlock($2c_i \rightarrow c_i$). Resolutions 64, 32, 16, 8 at the four levels and a $4 \times 4 \times 1024$ bottleneck; the bottleneck is the last down-conv's output with no mid-block.
- **Skip layout (a documented deviation).** The skip saved at level i is the *input* to that level's down block, so level $i+1$'s skip is level i 's encoder output after the strided conv—the standard pattern shifted down one level: the top decoder level receives `in_conv` features rather than a ResBlock output, and each skip is half the standard width. Tested (§11): the standard layout is neutral at 9.4 M parameters and unstable at 37 M at the default learning rate; the deviation is kept.
- **Region pooling** (§3.5) after the up blocks that produce 16×16 (256 channels) and 32×32 (128 channels).

3.5 Mask-guided region pooling

Let $h \in \mathbb{R}^{C \times H \times W}$ be the decoder feature map after an up block and $m_k \in [0, 1]^{H \times W}$, $k = 1..4$, the layout masks area-averaged to $H \times W$ for the regions *face*, *eyes*, *mouth* and *hair/background* = $1 - \text{face}$ (read directly from the network’s own input channels). The layer is

$$\bar{e}_k = \frac{\sum_p m_k(p) h(p)}{\sum_p m_k(p) + 10^{-6}} \in \mathbb{R}^C, \quad h' = h + \sum_{k=1}^4 m_k \otimes (W_k \bar{e}_k + b_k), \quad W_k \in \mathbb{R}^{C \times C}, b_k \in \mathbb{R}^C, \quad (4)$$

with $W_k = 0$, $b_k = 0$ at initialisation (the layer is the identity at step 0), all regions pooled from the same input h (order independent), implemented as three einsums. Parameters: $4(C^2 + C)$ per level, 0.33 M in total at width 64.

Interpretation. In attention terms this is masked attention with a single fixed query per region and uniform weights: the *routing* (which positions belong together) is supplied by the layout, and only the map on the aggregate is learned. Both irises receive an identical additive term derived from both of them, so the two local renderers start from the same offset and—with zero initialisation—can only be moved together. The closest published relative is SEAN (Zhu et al., 2020), which broadcasts a per-region style code taken from an encoder of a reference image; here the code is pooled from the network’s own decoder features, enforcing internal consistency rather than external style. The layer acts at 16×16 and 32×32 because that is where iris colour is decided; a global vector at the 4×4 bottleneck and attention there were both tried and did not move the defect (§11).

History of the definition. The first implementation updated h sequentially over regions (later regions pooled features already offset by earlier ones; regions overlap, as the eyes lie inside the face hull). The parallel form (4) differs from it by a mean $|\Delta \hat{x}|$ of 0.008 (1.6% of $|\hat{x}|$) on a trained checkpoint; the positive result was re-established on the parallel form before the final run (§12.1).

3.6 Layout prior

Unconditional generation requires a layout that comes from no real image. The prior (data/layouts.py, fitted by experiments/landmark_prior.py) is a point-distribution model (Cootes et al., 1995) with a Gaussian-mixture density.

Parametrisation. For a landmark set $L \in \mathbb{R}^{28 \times 2}$: pose $(c_x, c_y, \log s, \vartheta)$ with c the centroid, s the inter-ocular distance (between the two eye centroids) and ϑ the angle of the eye line; shape = the points centred, rotated by $-\vartheta$ and divided by s , flattened to $\mathbb{R}^{56}$. The map is exactly invertible (round-trip error 2×10^{-7}).

Mirror augmentation. $x \rightarrow -x$ with the point permutation that reverses every horizontal run and swaps the left/right groups ([4, 3, 2, 1, 0, 10, 9, 8, 7, 6, 5, 19, 18, 17, 22, 21, 20, 13, 12, 11, 16, 15, 14, 23, 26, 25, 24, 27, 28]). A wrong pairing shows as a large residual between the mean layout and the mean mirrored layout; the verified permutation gives a maximum per-point residual of 0.085 (the remainder is genuine dataset asymmetry: the image-left eye is systematically smaller, 0.356 vs 0.383, i.e. slightly turned faces).

Fit. On the 20,519 layouts with mean confidence ≥ 0.7 , doubled by mirroring: PCA on the shape vectors keeping 99% of variance (32 components), then a Dirichlet-process Gaussian mixture (variational, up to 16 components, full covariances, 500 iterations;

Table 1: Layout prior: marginal statistics (mean / std, $[-1, 1]$ units) of real layouts, DP-GMM samples and a KDE baseline (real layouts + Gaussian jitter, $\sigma = 0.01$).

measure	real	DP-GMM	KDE
eye spacing	0.769 / 0.055	0.768 / 0.054	0.769 / 0.057
eye size A / B	0.356 / 0.383	0.367 / 0.369	0.369 / 0.369
openness A / B	0.604 / 0.564	0.583 / 0.582	0.587 / 0.588
openness A – B	0.137 / 0.131	0.131 / 0.122	0.148 / 0.140
mouth width	0.185 / 0.115	0.185 / 0.108	0.182 / 0.113
face width	1.424 / 0.092	1.422 / 0.091	1.422 / 0.092
off-canvas fraction	0.003	0.001	0.004

`sklearn.BayesianGaussianMixture`) on [pose; PCA coefficients]; 16 components remain active. Sampling needs only numpy: choose a component by weight, draw from its Gaussian, invert the PCA and the pose normalisation, rasterise as in §2 (out-of-canvas points clipped only at rasterisation).

Checks (4,000 samples each; Table 1): the sampled marginals match the real ones, including the left/right openness *difference*, and a jitter-the-real-layouts baseline gives the same picture. Grids of sampled masks are plausible by eye, including the closed-eye mode.

3.7 Optimisation

Adam with global-norm gradient clipping at 1.0; batch 128; learning rate 10^{-3} peak with a linear warm-up over 500 steps (clamped to 10% of short runs) and a cosine decay to 10^{-5} at the last step (`optax.warmup_cosine_decay_schedule`). Weights published for sampling and evaluation are an exponential moving average with decay 0.999, warmed up as $\min(0.999, (1 + s)/(10 + s))$ at step s so early averages are not dominated by the initialisation (the scheme of the DDPM reference code, [Ho et al., 2020](#); the decay is shorter than the literature’s 0.9999 because these runs are tens of thousands of steps, not hundreds of thousands). Seed 49 for every run; the dataset order, noise and t draws are all derived from it.

The schedule is a late addition: every earlier run used a *constant* 10^{-3} , which turned out to be at the edge of stability for the 37 M model. Three runs with an added module diverged in a single epoch after tens of epochs of steady descent (§11, §13); the plain model survived 200 epochs with a thin margin. Constant 3×10^{-4} gives a *lower* training loss but a *worse* FID at 30 epochs (63.9 vs 56.5), so a lower constant rate was not adopted; warm-up plus decay was.

4 Evaluation protocol

FID. Inception pool features (`fidax`), 5,000 generated images against statistics of the full 21,551-image set. Real-vs-real at 5,000 samples gives a floor of 2.1. **Noise:** two seeds of the same 30-epoch configuration scored 56.5 and 52.7; differences under ≈ 5 FID at short horizons are not evidence. **Sampler:** on the 200-epoch model, 16 / 32 / 64 Dopri5 steps score 31.6 / 31.4 / 32.0; training-time and screening FID therefore use 16 steps at a quarter of the cost. Training batch 256 is *slower* per epoch than 128 (39.9 vs 29.0 s) on this GPU;

128 is kept.

Loss as the sensitive statistic. Across seeds the 30-epoch training loss varies by ± 0.0003 while FID varies by ± 4 ; for architecture changes that leave the objective unchanged, loss is the sharper screen.

Precision / recall. Kynkäänniemi et al. (2019), $k = 3$ nearest neighbours in Inception space, 5,000 vs 5,000; real-vs-real ceiling 0.774 / 0.765.

Iris-mismatch rate. Split the dataset-mean eye mask at the vertical midline into a left and a right region. For an image, take the mask-weighted mean chroma (C_b, C_r) (BT.601, from $[-1, 1]$ RGB) of each region and their Euclidean distance. A sample is “mismatched” if that distance exceeds the real data’s 95th percentile (0.1027), so real data scores $\approx 5\%$ by construction (two disjoint halves: 5.1% and 5.9%). Validated by inspection: the 32 most-flagged samples of the 200-epoch model are all genuine red/blue, yellow/blue, purple/green pairs in otherwise well-formed faces; the least flagged are matched. **This metric is uncorrelated with FID:** the best-FID unconditioned model has a 35.5% rate.

Local-detail proxies. Mean Sobel magnitude (“edge mass”) and mean ink map of samples relative to real images; and, for the bottleneck analysis, per-region prediction error and per-cell texture statistics (§10).

5 Auxiliary losses on $\hat{x}$: theory

The shipped model added to (2) an edge loss and (optionally) a line loss:

$$\mathcal{L}_{\text{sobel}} = \frac{1}{2} (\mathbb{E} |\nabla_x \hat{x} - \nabla_x x_1| + \mathbb{E} |\nabla_y \hat{x} - \nabla_y x_1|), \quad \text{Sobel gradients per channel, L1 (Mathieu et al., 2016),} \quad (5)$$

$$\mathcal{L}_{\text{line}} = \mathbb{E} |\text{ink}(\hat{x}) - \text{ink}(x_1)|, \quad \text{ink}(x) = \text{relu}(\text{softclose}_{3 \times 3}(\text{luma } x) - \text{luma } x), \beta = 10, \quad (6)$$

with weights 0.1 and 0 respectively.

Proposition 1 (Irreducibility). *Let $x^*(t) = \mathbb{E}[x_1 \mid x_t]$ be the minimiser of (2). For any $\mathcal{L}_{\text{aux}}(\hat{x}, x_1) \geq 0$ with $\mathcal{L}_{\text{aux}}(x_1, x_1) = 0$, the value $\mathcal{L}_{\text{aux}}(x^*(t), x_1)$ is in general strictly positive and cannot be reduced by any network without leaving the optimum of (2). Only the excess $\mathcal{L}_{\text{aux}}(\hat{x}) - \mathcal{L}_{\text{aux}}(x^*)$ is a usable signal.*

At low t , x^* is a blurry average; an edge or line loss against the sharp x_1 measures that blur, and its gradient at x^* points toward “sharper than the evidence supports”. Whether this is *bias* rather than harmless noise depends on the minimiser of the auxiliary term: L1 on gradient differences is minimised by the per-pixel *median* of the target gradient, which under uncertainty about edge position is nearer zero than the mean, so minimising an irreducible L1 penalty rewards *flatter* predictions (hedging). The line loss is L1 on a nonlinear map and is biased for that reason too. Section 6 estimates the reducible fraction empirically; §7 confirms the prediction in training.

Table 2: Discrimination: loss under a degradation, normalised by the loss against an unrelated image.

degradation	pixel L2	sobel	line
blur $\sigma = 1$	0.064	0.379	0.552
noise $\sigma = 0.05$	0.005	0.112	0.103
desaturate 50%	0.019	0.075	0.000
brightness +0.2	0.073	0.000	0.000
erase strokes	0.097	0.355	0.550
jitter 1 px	0.206	0.625	0.812

Proposition 2 (Weighted losses). *For a per-pixel weight w and prediction c at fixed x_t ,*

$$J(c) = \mathbb{E}[w(c - Y)^2 \mid x_t], \quad \nabla J = 0 \Rightarrow c^* = \frac{\mathbb{E}[wY \mid x_t]}{\mathbb{E}[w \mid x_t]} = \mathbb{E}[Y \mid x_t] + \frac{\text{Cov}(w, Y \mid x_t)}{\mathbb{E}[w \mid x_t]}.$$

$c^* = \mathbb{E}[Y \mid x_t]$ iff w is conditionally independent of Y given x_t , i.e. a function of (t, x_t) only.

Proof sketch. Tower property, condition on the argument of the predictor (not on Y : conditioning on Y would “derive” $c = Y$, which no function of x_t can be); J is a strictly convex quadratic in c ; differentiate and expand $\mathbb{E}[wY] = \mathbb{E}[w]\mathbb{E}[Y] + \text{Cov}(w, Y)$. Equivalently, conditioning on the pair (x_0, x_1) : a weight $w(x_0, x_1)$ is the unweighted CFM loss under the re-weighted coupling $\tilde{p} \propto w p$, so by Lipman et al. (2023) (Thm. 2) the learned field transports noise to the data distribution *tilted by w* . The per-sample gradient factorisation $W \odot r$ is correct but does not make the original optimum stationary, because $\mathbb{E}[wr \mid x_t] = \text{Cov}(w, r \mid x_t) \neq 0$ for a stochastic coupling ($r = 0$ for every sample is unattainable). Weights of the form $\lambda(t)$ —the $1/(1-t)^2$ factor, any t -gate—are free; weights built from x_1 (Sobel magnitude, ink map, region masks) are not, unless the map is nearly constant across the posterior (true for the eye mask on aligned faces, false for edge maps at low t).

6 Offline tests (no training)

All on the 100-epoch 9.4 M checkpoint that was trained *with* the edge loss (a caveat noted below), 256 images, seed 0 (experiments/aux_signal.py on the archive branch).

Discrimination. $\mathcal{L}(x, d(x))/\mathcal{L}(x, x')$ for degradations d and a random other image x' (Table 2). Both losses are 5–6 $\times$ more sensitive than pixel L2 to stroke erasure (a 3×3 luma closing) and to a 1-px jitter, and blind to global colour: they measure what they claim.

Pixel-space descent. From $\text{blur}_1(x)$, 200 Adam steps on the pixels minimising $\text{L2} + w \mathcal{L}_{\text{aux}}$: L2 alone recovers ink mass by step 25 and 113 dB PSNR by step 200; every auxiliary term is slower (sobel 0.1: 60 dB; line 1.0: 47 dB). With a known target the terms carry nothing L2 lacks.

Table 3: Irreducibility estimate: auxiliary loss of $\hat{x}(t)$ versus that of the PSNR-matched blur $\text{blur}_{\sigma^*}(x_1)$, and the reducible fraction.

t	σ^*	sobel: $\hat{x}$ / blur	line: $\hat{x}$ / blur	reducible sobel / line
0.10	8.0	0.807 / 0.820	0.099 / 0.101	0 / 0
0.30	2.5	0.677 / 0.744	0.090 / 0.098	0 / 0
0.50	1.1	0.496 / 0.516	0.071 / 0.085	0 / 0
0.65	0.7	0.367 / 0.344	0.054 / 0.068	0.06 / 0
0.85	0.5	0.205 / 0.184	0.029 / 0.041	0.10 / 0

Table 4: Loss ablations, 9.4 M model, 40 epochs. “Class” is the verdict of Prop. 2.

arm	FID@39	ink/real	edge/real	class
no auxiliary (reference)	68.0	0.79	0.86	—
sobel L1, weight 0.1 (shipped default)	74.7	0.68	0.76	biased (median)
sobel L2, weight 0.05	68.0	—	—	unbiased (linear map) – inert
FM weight 1 + $ \text{Sobel}(x_1) /\text{mean}$	65.3	0.89	0.91	biased (tilt)
FM weight from $ \text{Sobel}(\text{sg } \hat{x}) $	65.3	0.79	0.87	unbiased
FM weight 1 + $\text{ink}(x_1)/\text{mean}$	97.0			biased; $\sim 50\times$ per pixel
FM weight 1 + m_{eye} , coef. 1.0 / 0.2	117.7 / 76.0			$\sim$ unbiased; $20\times / 5\times$

Irreducible fraction. For each t on a 0.05 grid, match the checkpoint’s $\hat{x}(t)$ with a Gaussian blur of x_1 at equal PSNR and read the blur’s auxiliary loss as an estimate of $\mathcal{L}_{\text{aux}}(x^*)$ (Table 3). The line loss of $\hat{x}$ is *below* the matched blur at every t ; the sobel loss is within $\pm 5\%$ except a 5–10% excess for $t \in [0.6, 0.85]$. At every t at least 90% of both penalties is unreachable, and no t -window contains signal worth gating for. With the default weights the auxiliary penalty is $0.6\text{--}1.1\times$ the velocity loss for $t < 0.5$. Caveats: a Gaussian blur is not $\mathbb{E}[x_1 \mid x_t]$ (it likely *over*-states the irreducible part), and the checkpoint was trained with the edge loss; hence the ablations of §7.

Gradient geometry. Cosine similarity of per-sample gradients with respect to $\hat{x}$: $\cos(\nabla \text{L2}, \nabla \text{sobel}) \approx 0.65$, $\cos(\nabla \text{L2}, \nabla \text{line}) \approx 0.40$ at every point tested. The sobel-L1 gradient is $\text{Sobel}^\top \text{sign}(\cdot)$: a saturated ± 1 stripe pattern over the whole image including flat background.

7 Training ablations of the losses

9.4 M model, seed 49, 40 epochs, FID every 10 unless stated.

Shipped default vs off (paired). Table 4: turning the edge loss *off* is better at every evaluation and the model trained *with* it generates *fewer* edges (edge mass 0.76 vs 0.86 of real; ink 0.68 vs 0.79). On training-time interpolants the two models agree within 1.5% ($\mathcal{L}_{\text{sobel}}(\hat{x}, x_1)$ at $t = 0.1$: 0.808 vs 0.821)—the sliver §6 called reducible—so the difference is how hedged predictions compound along the sampling trajectory. Decision: default 0.

Variants. The L2 Sobel term is exactly inert (68.0 vs 68.0), as predicted for an unbiased term with $\leq 10\%$ reducible signal. The two edge re-weightings end at the same FID whether

or not they carry the covariance term of Prop. 2; the tilt shows only in the proxies (edgier samples). Sparse-map weights (ink, eye) are harmful at every coefficient tried because the batch-mean normalisation turns “average extra weight 1” into $20\text{--}50\times$ on a few percent of pixels; the eye weight does reduce iris mismatch dose-dependently ($41.8 \rightarrow 37.5 \rightarrow 32.8\%$) at a prohibitive FID cost.

At length. 37 M model, 200 epochs, FID at 49/99/149/199: plain 43.1/36.3/36.5/32.0; with the unbiased edge re-weighting 42.9/37.7/35.9/33.4. The one loss-side candidate is a wash. A mask-prediction auxiliary head on the encoder (BCE on area-pooled masks, weights 0.25 and 1.0) was monotonically worse at 9.4 M over 100 epochs (57.6, 69.7 vs 53.5 best) and at 37 M (62.4 vs 56.5). **The pixel-space auxiliary family is closed** at both widths and all horizons tested.

8 Scaling and the noise floor

Width and length are the two levers that moved FID substantially: 9.4 M / 40 epochs 68.0; 9.4 M / 100 epochs 53.5; 37 M / 30 epochs 56.5; 37 M / 200 epochs 32.0, still falling ($36.5 \rightarrow 32.0$ over the last 50). A spectral check supports spending on width: the effective rank (entropy of the normalised singular values of each convolution reshaped to $(\text{out}, \text{in} \cdot k^2)$) is 0.88 of the maximum at 9.4 M and 0.85 at 37 M, i.e. the doubled width is used, not idle; from 30 to 200 epochs the stable rank rises ($13.8 \rightarrow 17.5$) while the entropy rank falls slightly ($0.853 \rightarrow 0.825$), the signature of training concentrating a few directions while spreading energy into the mid-spectrum. Curation (§2) gave 53.2 / loss 0.0875 against 56.5–52.7 / 0.0890 at 30 epochs.

9 Sampling-time edge guidance

Priors about *samples* belong at sampling time, where they cannot bias the learned field. With $E(\hat{x}) = \text{relu}(m_{\text{real}} - |\text{Sobel}(\hat{x})|)^2$ (the one-sided deficit against the data’s mean edge mass, $m_{\text{real}} = 1.30$),

$$g = \frac{\partial E(\hat{x}(x_t, t))}{\partial x_t}, \quad \hat{x}' = \hat{x} - \lambda \mathbb{I}[t_{\text{lo}} < t < t_{\text{hi}}] \frac{g}{\text{rms}(g)}, \quad v = \frac{\hat{x}' - x_t}{\max(1 - t, \epsilon_d)}, \quad (7)$$

gradient through the network, RMS-normalised per sample. Results (5,000-sample FID): 9.4 M model $67.4 \rightarrow 59.0$ at $\lambda = 0.02$ ($\lambda = 0.05/0.10/0.20$: 59.4/62.1/69.5, overshooting as edge mass passes real); 37 M 200-epoch model $31.4 \rightarrow 28.1$. Restricting guidance to a t -interval (Kynkäänniemi et al., 2024) helps slightly: on the 9.4 M model $\lambda = 0.05$ on $t \in (0.5, 1)$ gives 57.8, while $(0, 0.5)$ is the worst window (61.4). Guidance raises precision ($0.52 \rightarrow 0.58$) and leaves recall at 0.15: a mode-seeking correction, which bounds its value at a few FID. Cost: one backward pass per network evaluation.

10 Where the model fails

On the 37 M 200-epoch unconditioned model (`experiments/bottleneck.py`).

Kind of gap. FID floor 2.1. Precision 0.521, recall **0.154** (ceiling 0.774/0.765). Coverage dominates: 85% of real images have no generated neighbour. The samples farthest from the real manifold are washed-out, low-contrast images with incoherent hair, clothing and background—averages of modes the model has not learned, not broken faces.

Where. Prediction error divided by per-pixel data variance (512 real pairs): at $t = 0.3$, face 0.41, eyes 0.43, **mouth** 0.50, hair/background 0.25; the mouth is recovered worst relative to what is predictable because small low-variance high-frequency features receive the least loss weight. Generated-vs-real local statistics on an 8×8 grid: edge mass -9 to -12% , luma std -9 to -13% (worst at the mouth/nose centre, -30 to -40%), chroma std -16 to -21% everywhere. Colour *means* are right (skin chroma $-0.062/0.089$ vs $-0.070/0.098$); colour *spread* is 13% narrow and the chroma-histogram distance to real is $2.1 \times$ the real-vs-real floor. A palette loss therefore has no target; what is missing is diversity, which no per-sample loss supplies.

Eyes. Iris mismatch 35.5% on this model (§4), i.e. a long-range colour-coupling failure that FID does not see and that motivated §12.

11 Architecture variants that did not help

All paired, 37 M, 30 epochs unless stated; loss (stable to ± 0.0003) is the primary read-out.

- **Standard skips.** 9.4 M: 76.0 vs 76.0 (neutral). 37 M at constant 10^{-3} : FID 188, loss flat at 0.108 from epoch 3, samples frozen into a tiled texture—an optimisation collapse; at 3×10^{-4} : 63.6 with loss $0.0874 < 0.0890$ but no better than the legacy layout. Kept legacy.
- **Mid-block (ResBlock–attention–ResBlock at 4×4).** 9.4 M, 40 epochs: 78.7 vs 77.1. 37 M, 200 epochs, constant 10^{-3} : 75.8 with a loss spike to 0.635 at epoch 12—an optimisation failure, not a verdict on attention. It had the lowest iris mismatch of the unconditioned models (23.0%), consistent with global mixing helping the coupling.
- **Global code.** $g = W_2 \text{SiLU}(W_1 \text{LN}(\text{mean}_{hw} z) + b_1) + b_2$ from the bottleneck z , $W_2 = 0$ at init, added to the decoder’s time embedding (the ADM/DiT class-embedding recipe). Loss 0.0869 vs 0.0870, FID 63.8 vs 63.9, mismatch 43.8% vs 42.6%. The path *did* switch on ($\|g\| \approx 0.6\text{--}0.8 \|e\|$, image-dependent) and changed nothing: the decoder is not short of global *information*; the defect is in how two regions are *rendered*, which is what §3.5 addresses.

12 Layout conditioning and region pooling: results

12.1 Conditioning alone

37 M, curated. With dropout 0.15 (30 epochs): unconditional mode 62.6, prior layouts 57.8, real layouts 57.2, mismatch 29.7%; without dropout (40 epochs, loss 0.0813): prior 50.7, real 50.1, mismatch 33.6%. Unconditional references at 30 epochs: 56.5/52.7/53.2. Layout information lowers the training loss but not FID (real and prior layouts agree, so the prior is

not the limitation) and does not touch iris agreement: a layout says where the eyes are, not what colour they share. Dropout costs ≈ 7 FID at this horizon.

12.2 Region pooling

Same recipe (pure conditional, 40 epochs, LR 10^{-3} constant, seed 49, 16-step FID):

40 epochs	loss	FID prior / real layouts	iris mismatch prior / real
no region pool	0.0813	50.7 / 50.1	33.6% / 38.7%
region pool (sequential form)	0.0807	48.2 / 48.0	6.6% / 5.1%
region pool (parallel form, confirmation)	0.0815	52.1 / 51.4	6.25% / 5.1%
real data			5.0–5.9%

The mismatch rate falls to the data’s own rate. Paired samples (same noise, same layouts) are the same face pair-for-pair with only the iris pairs changed—matched and more saturated. The FID difference between the two region-pool runs (48.2 vs 52.1) is inside the single-seed noise. The eye-region chroma distribution of the region-pool model matches real data in spread (saturation std 0.067 vs 0.071; C_b , C_r std 85–95% of real) and in hue histogram (same dominant blue-green band, 32% in both), so the layer enforces *agreement*, not a particular colour.

13 Final training run

Recipe (just paper): 37 M U-Net with region pooling, pure conditional on the three-channel layout, curated data, batch 128, EMA 0.999, Dopri5 16-step FID every 50 epochs with prior-sampled layouts, 300 epochs in total.

Course of the run. The first attempt ran at a constant 10^{-3} and diverged in a single epoch at epoch 80 (loss 0.0784 $\rightarrow$ 0.2837, samples destroyed, partial recovery to 0.107), after a steady descent to 0.0776 and FID 42.0 at epoch 49. The warm-up-plus-cosine schedule of §3.7 was introduced in response and the run resumed from the epoch-49 EMA weights with a fresh optimiser (Adam moments are not checkpointed). A second segment of 49 epochs (loss 0.0770) ended in a `cuSolver` failure inside the FID computation caused by a concurrent GPU job—an operational error, not a modelling one—and the run resumed again from those weights for the final 200 epochs. The effective schedule is three segments (49 constant, 49 and 200 warm-up-cosine), with the learning rate near its peak at both cut points, so it approximates one long cosine; checkpoints and loss logs of every segment are kept (`runs/wide_rp_300_ep49.eqx`, `_ep98.eqx`, and the corresponding `losses.json` files).

Results. Final loss 0.0557. Training-time FID over the last segment (16-step sampler, prior layouts), at cumulative epochs $\approx 147/197/247/297$: 47.8/34.9/36.2/37.5—best at ≈ 200 cumulative epochs, drifting up thereafter; `best_model.eqx` is that checkpoint. Final evaluation of the last weights (5,000 samples, 16 steps): FID **37.8** with prior layouts and 34.2 with real layouts; **iris mismatch** 4.7% under either (real data 5.0%; mean left/right

wide_rp_300: 200 epochs, base_channels=64, cond_channels=3, region_pool=1, final FID 37.5



Figure 1: Showcase page produced by `generate_figures.py`: loss and FID curves, samples with layouts from the prior, layout-to-image pairs, real vs generated, noise interpolation at a fixed layout, and eye-region crops.

chroma distance 0.039 vs 0.037 real). Layout adherence is strong (Fig. 1, layout-to-image panel: tilt, eye spacing and eye height track the mask in every sample). Against the plain unconditioned 200-epoch model (31.4 at the same sampler) the conditioned model is ≈ 6 FID behind at the end and ≈ 3.5 behind at its best point, while solving the eye defect outright; the prior-vs-real layout gap (3.5 FID), absent at 40 epochs (50.7 vs 50.1), opened as adherence sharpened, which points at a residual mismatch between the prior’s layouts and the data’s as one component of the deficit. Two confounds remain open: the two warm restarts in the schedule, and the absence of a schedule-matched unconditioned control; the precision/recall of the final checkpoint was not measured (the guided stage of the analysis exceeded GPU memory at the batch size used). Off-the-shelf anime super-resolution models (Real-ESRGAN variants, APISR) were tried on the samples for presentation and dropped: trained on clean-then-degraded pairs, they render the generator’s own errors as confident detail, and the native 64×64 samples read better.

14 Decisions and their evidence (summary)

- **x-prediction, velocity-space loss, floor 0.05, logit-normal t :** JiT recipe; the parametrisation is what makes losses on $\hat{x}$ possible, and §5 is why none are used.
- **No auxiliary losses:** Prop. 1–2, Tables 3–4, the 200-epoch wash. Closed at both widths.
- **Guidance instead:** -3 to -8 FID, $\lambda \approx 0.02$, middle/late window; reported separately from unguided numbers.
- **GroupNorm (8 groups), zero-init adaLN, no output gate, no final activation, time scale 10^3 :** documented in code; the zero-init adaLN is why every added module (region pool, global code) can be identity at initialisation.
- **Legacy skips kept:** neutral at 9.4 M, unstable at 37 M.
- **Width 64, batch 128, 16-step evaluation, curation on:** §8, §4.
- **Warm-up + cosine:** three single-epoch divergences at constant 10^{-3} .
- **Pure conditional (no dropout):** dropout ≈ 7 FID at the screening horizon; the prior makes the unconditional mode unnecessary.
- **Region pooling at 16×16 and 32×32 , four regions, zero-init, parallel form:** §12; the only structural change with a reproduced, metric-validated win.
- **Layout prior: PDM + PCA(99%) + DP-GMM:** Table 1; sampling is numpy-only.

Open. Coverage (recall 0.15) is the largest remaining gap and moves with steps and capacity; EDM-style augmentation conditioning with hair/eye hue rotation (a dataset-specific coverage prior, semantically valid only for this domain), spatial mixing at 16×16 with warm-up, a hair-colour consistency metric for the region-pool layer, flip-equivariant sampling, and re-noise-and-re-solve refinement of finished samples with the model’s own prior (SDEdit, Meng et al., 2022; Restart sampling, Xu et al., 2023; implemented in `experiments/refine.py`, results pending) as the in-distribution alternative to external upscalers. Shelved with reasons recorded: structured output heads (line/fill, palette) as precision-side bets with a representational risk.

Reproducibility

Repository: `tinyflow`, branch `feat/pixel-unet`; the complete working tree of the campaign, including every dropped variant and the offline harness, is preserved on branch `exp/aux-loss-campaign`. `experiments/METHODS.md` holds every table of this paper with the commands that produced it; `just paper NAME 300 50` reproduces the final run and the figures. Hardware: one NVIDIA RTX 4090 (24 GB); training throughput ≈ 22 s per epoch (166 batches of 128) for the 37 M model; a 16-step 5,000-sample FID takes ≈ 2 minutes.

References

T. F. Cootes, C. J. Taylor, D. H. Cooper, J. Graham. Active shape models—their training and application. *CVIU*, 1995.

- J. Ho, A. Jain, P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- J. Ho, T. Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.
- T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila. Improved precision and recall metric for assessing generative models. *NeurIPS*, 2019.
- T. Kynkäänniemi et al. Applying guidance in a limited interval improves sample and distribution quality in diffusion models. *NeurIPS*, 2024.
- T. Li, K. He. Back to basics: let denoising generative models denoise (JiT). *CVPR*, 2026.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow matching for generative modeling. *ICLR*, 2023.
- C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, S. Ermon. SDEdit: guided image synthesis and editing with stochastic differential equations. *ICLR*, 2022.
- M. Mathieu, C. Couprie, Y. LeCun. Deep multi-scale video prediction beyond mean square error. *ICLR*, 2016.
- W. Peebles, S. Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- Y. Wu, K. He. Group normalization. *ECCV*, 2018.
- Y. Xu, M. Deng, X. Cheng, Y. Tian, Z. Liu, T. Jaakkola. Restart sampling for improving generative processes. *NeurIPS*, 2023.
- P. Zhu, R. Abdal, Y. Qin, P. Wonka. SEAN: image synthesis with semantic region-adaptive normalization. *CVPR*, 2020.